WEEK 4 TASK BY RIDHI MALHOTRA

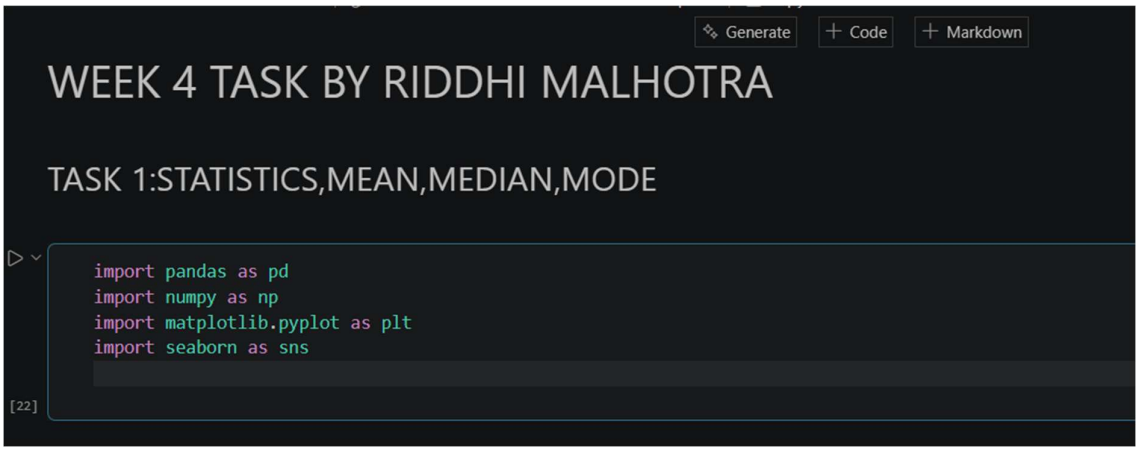
----------TASK 1----------

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns



We need to import statistics library to use statistics functions

from statistics import mean, median, mode

data=[10,20,30,30,40,50,50,10,10,10]

print("DATASET:",data)

#Mean → The average value, showing the central tendency of the dataset.

mean=mean(data)

print("\nMEAN: ",mean)

#Median → The middle value, less affected by extreme values/outliers.

median=median(data)

print("\nMedian: ",median)

#Mode → The most frequent value, showing the most common observation.

mode=mode(data)

print("\nMode: ",mode)

##This task introduces the three measures of central tendency — Mean, Median, and Mode.

#The mean represents the average score, giving an overall idea of performance.

#The median identifies the middle score, which is less affected by extreme values and shows the central point of the data.

#The mode highlights the most frequently occurring score, indicating the most common observation.

#Together, these measures provide a comprehensive understanding of the dataset's distribution.

```
## We need to import statistics library to use statistics functions
from statistics import mean, median, mode

data=[10,20,30,30,40,50,50,10,10,10]

print("DATASET:",data)

#Mean → The average value, showing the central tendency of the dataset.
mean=mean(data)
print("\nMEAN: ",mean)

#Median → The middle value, less affected by extreme values/outliers.
median=median(data)
print("\nMedian: ",median)

#Mode → The most frequent value, showing the most common observation.
mode=mode(data)
print("\nMode: ",mode)

##This task introduces the three measures of central tendency — Mean, Median, and Mode.
#The mean represents the average score, giving an overall idea of performance.
#The median identifies the middle score, which is less affected by extreme values and shows the central point of the data.
#The mode highlights the most frequently occurring score, indicating the most common observation.
#Together, these measures provide a comprehensive understanding of the dataset's distribution.
```

[2]

```
DATASET: [10, 20, 30, 30, 40, 50, 50, 10, 10, 10]

MEAN:  26

Median:  25.0

Mode:  10
```

----------TASK 2----------

```
data=[10,20,30,30,40,50,50,10,10,10]

#Variance → Measures how far values spread from the mean.

variance=np.var(data)

print("Variance:",variance)


#Standard Deviation → Indicates the average distance of values from the mean.

standard_deviation=np.std(data)

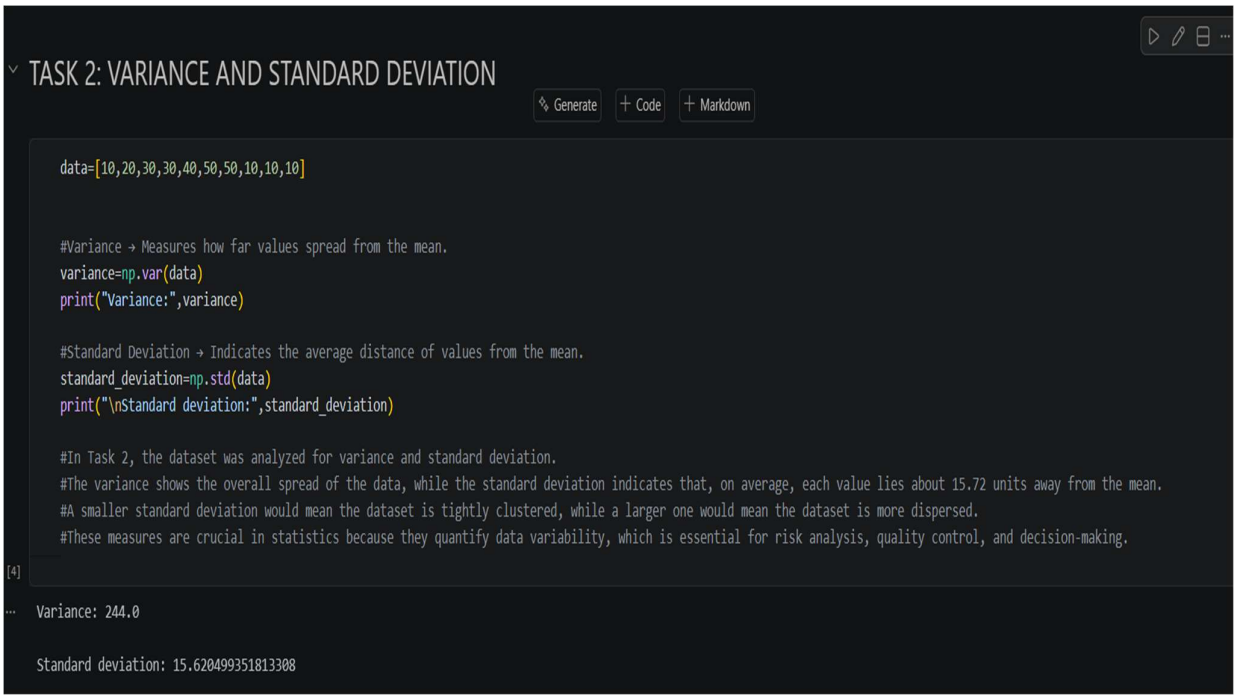
print("\nStandard deviation:",standard_deviation)


#In Task 2, the dataset was analyzed for variance and standard deviation.

#The variance shows the overall spread of the data, while the standard deviation indicates that, on average, each value lies about 15.72 units away from the mean.

#A smaller standard deviation would mean the dataset is tightly clustered, while a larger one would mean the dataset is more dispersed.

#These measures are crucial in statistics because they quantify data variability, which is essential for risk analysis, quality control, and decision-making.
```



----------TASK 3----------

PART 1: CORRELATION

```
# DATASET
age = [18, 22, 25, 30, 35, 40, 45]
spending = [2000, 2500, 2700, 3000, 3200, 3500, 3700]

# Calculate correlation coefficient
correlation = np.corrcoef(age, spending)[0,1]

print("Correlation between Age and Monthly Spending:", correlation)

# Interpretation
if correlation > 0.7:
   relation = "Strong Positive"
elif correlation > 0:
   relation = "Weak Positive"
elif correlation < -0.7:
   relation = "Strong Negative"
elif correlation < 0:
   relation = "Weak Negative"
else:
   relation = "No Correlation"

print("Relationship:", relation)
```

#Correlation measures the strength and direction of the relationship between two numerical variables.

Values range from −1 to +1:

#+1 → Perfect positive relationship (both increase together).

#−1 → Perfect negative relationship (one increases, the other decreases).

#0 → No relationship.

#As age increases, spending also increases. This is a strong positive correlation.

TASK 3: CORRELATION AND PROBABILITY BASICS

TASK 3 PART 1: CORRELATION

```
# DATASET
age = [18, 22, 25, 30, 35, 40, 45]
spending = [2000, 2500, 2700, 3000, 3200, 3500, 3700]

# Calculate correlation coefficient
correlation = np.corrcoef(age, spending)[0,1]

print("Correlation between Age and Monthly Spending:", correlation)

# Interpretation
if correlation > 0.7:
    relation = "Strong Positive"
elif correlation > 0:
    relation = "Weak Positive"
elif correlation < -0.7:
    relation = "Strong Negative"
elif correlation < 0:
    relation = "Weak Negative"
else:
    relation = "No Correlation"

print("Relationship:", relation)

#Correlation measures the strength and direction of the relationship between two numerical variables.
# Values range from −1 to +1:
    #+1 → Perfect positive relationship (both increase together).
```

[5]

```
    relation = "No Correlation"

print("Relationship:", relation)

#Correlation measures the strength and direction of the relationship between two numerical variables.
# Values range from −1 to +1:
    #+1 → Perfect positive relationship (both increase together).
    #−1 → Perfect negative relationship (one increases, the other decreases).
    #0 → No relationship.

#As age increases, spending also increases. This is a strong positive correlation.
```

[5]

```
Correlation between Age and Monthly Spending: 0.9835563367554027
Relationship: Strong Positive
```

PART 2: PROBABILITY

```
# Probability Example: Bag of balls

total_balls = 10

red_balls = 4


# Probability of drawing a red ball

prob_red = red_balls / total_balls


print("Probability of drawing a Red ball:", prob_red)


# Explanation

print(f"This means there is a {prob_red*100:.0f}% chance of drawing a red ball.")

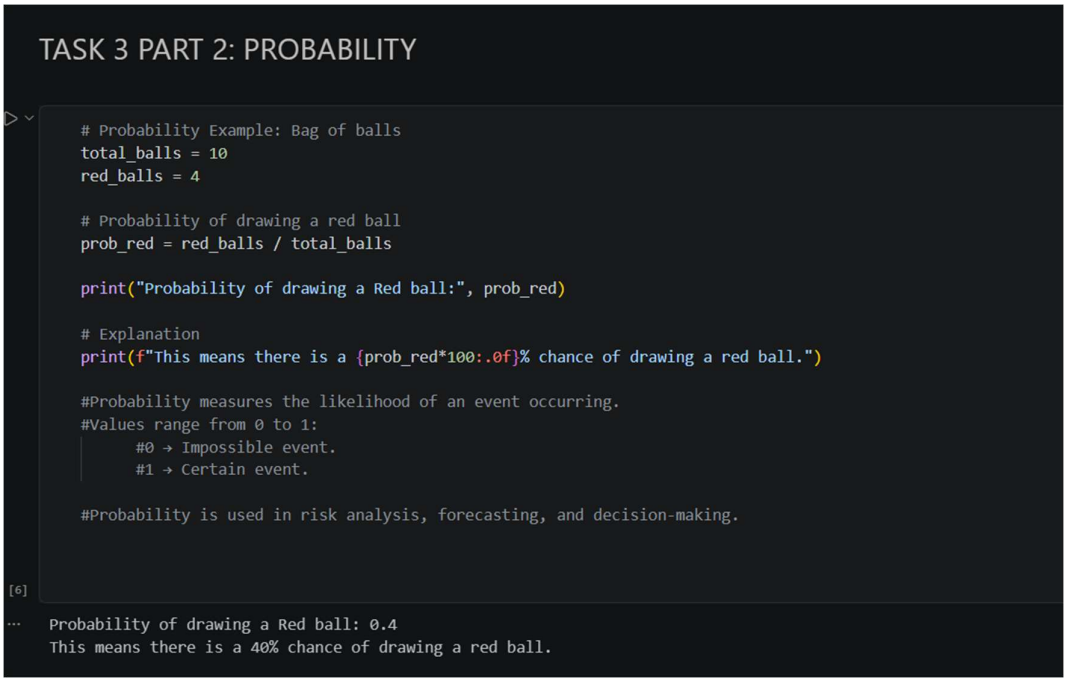
#Probability measures the likelihood of an event occurring.

#Values range from 0 to 1:

    #0 → Impossible event.

    #1 → Certain event.

#Probability is used in risk analysis, forecasting, and decision-making.
```



----------TASK 4----------

```
data=[10,12,20,24,18,16,15,19,25,100]


mean = np.mean(data)

std_dev = np.std(data)


# --- Method 1: Z-Score ---

outliers_z = [x for x in data if abs((x - mean) / std_dev) > 2]

print("Outliers using Z-Score:", outliers_z)
```

#Z-SCORE:Measures how far each data point is from the mean in terms of standard deviations.

```
# --- Method 2: IQR ---

Q1 = np.percentile(data, 25)

Q3 = np.percentile(data, 75)

IQR = Q3 - Q1


lower_bound = Q1 - 1.5 * IQR

upper_bound = Q3 + 1.5 * IQR


outliers_iqr = [x for x in data if x < lower_bound or x > upper_bound]


print("Outliers using IQR:", outliers_iqr)
```

#IQR:Uses percentiles to detect values that lie far outside the typical range.

#Outliers are unusual data points that lie far outside the typical range of values.

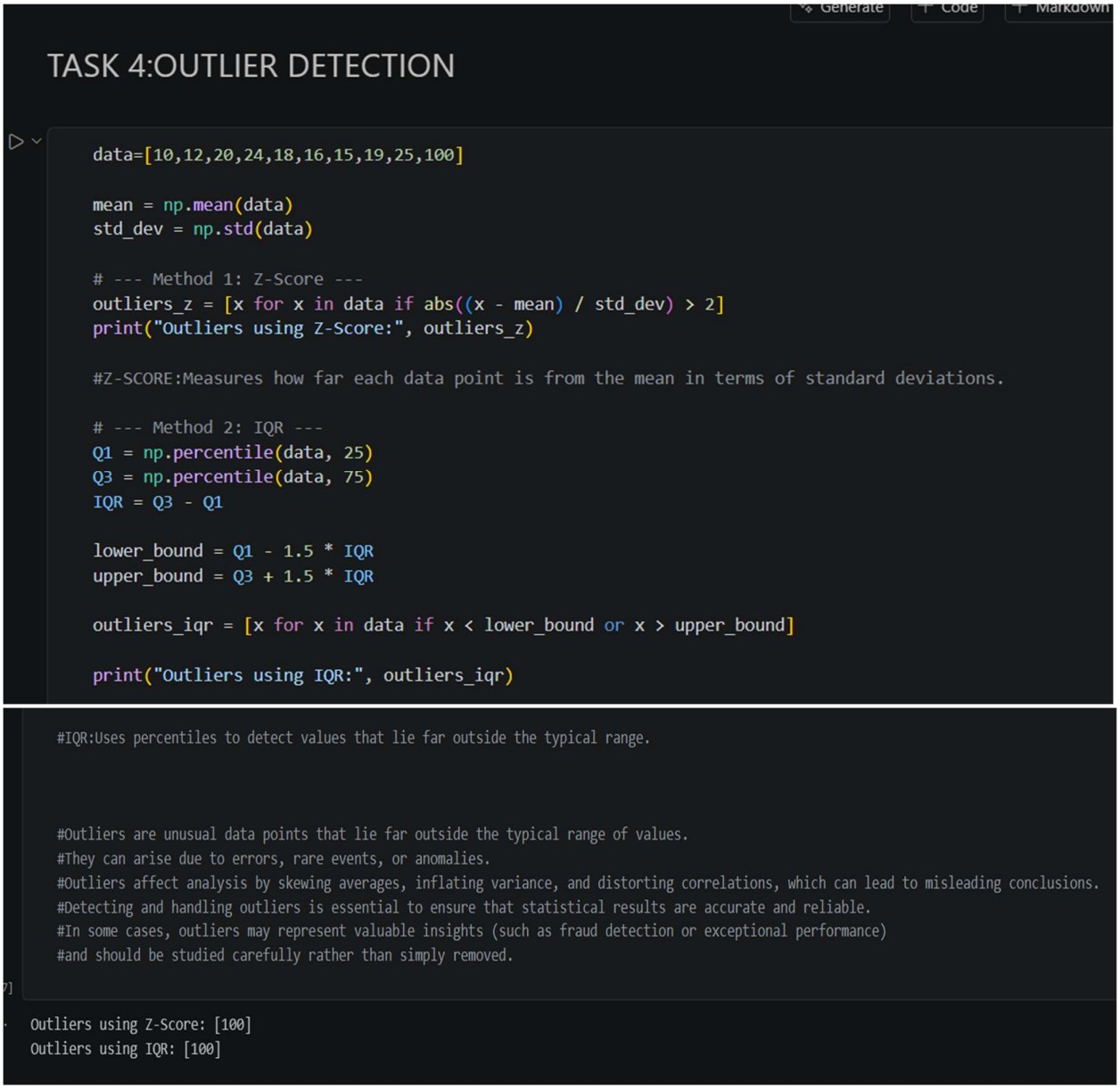
#They can arise due to errors, rare events, or anomalies.

#Outliers affect analysis by skewing averages, inflating variance, and distorting correlations, which can lead to misleading conclusions.

#Detecting and handling outliers is essential to ensure that statistical results are accurate and reliable.

#In some cases, outliers may represent valuable insights (such as fraud detection or exceptional performance)

#and should be studied carefully rather than simply removed.

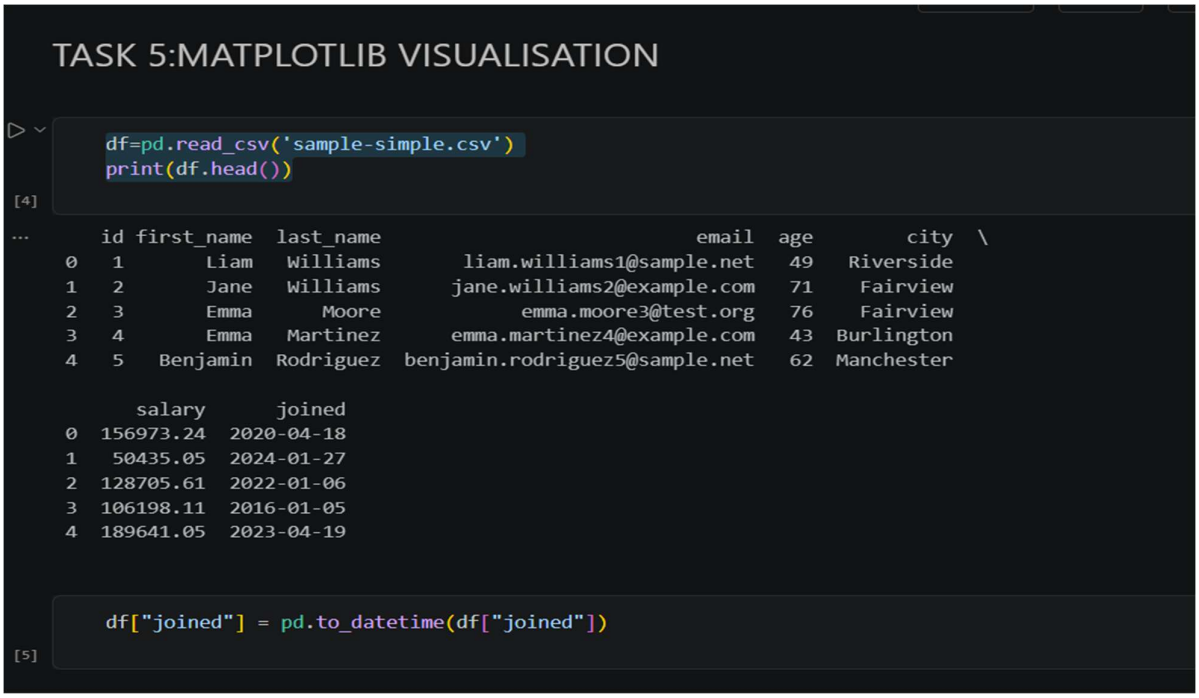


----------TASK 5----------

df=pd.read_csv('sample-simple.csv')

print(df.head())

df["joined"] = pd.to_datetime(df["joined"])



LINE CHART

df_sorted=df.sort_values("joined")

plt.plot(df_sorted["joined"],df_sorted["salary"],marker='*')

plt.title("SALARY TREND BY JOINING DATE")

plt.xlabel("JOINING YEAR")

plt.ylabel("SALARY DISTRIBUTION")

plt.show()

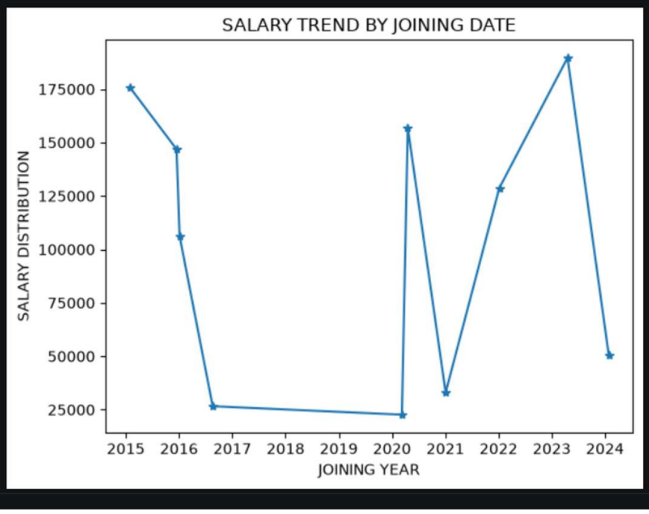
#Line Chart – Salary Trend by Joining Date

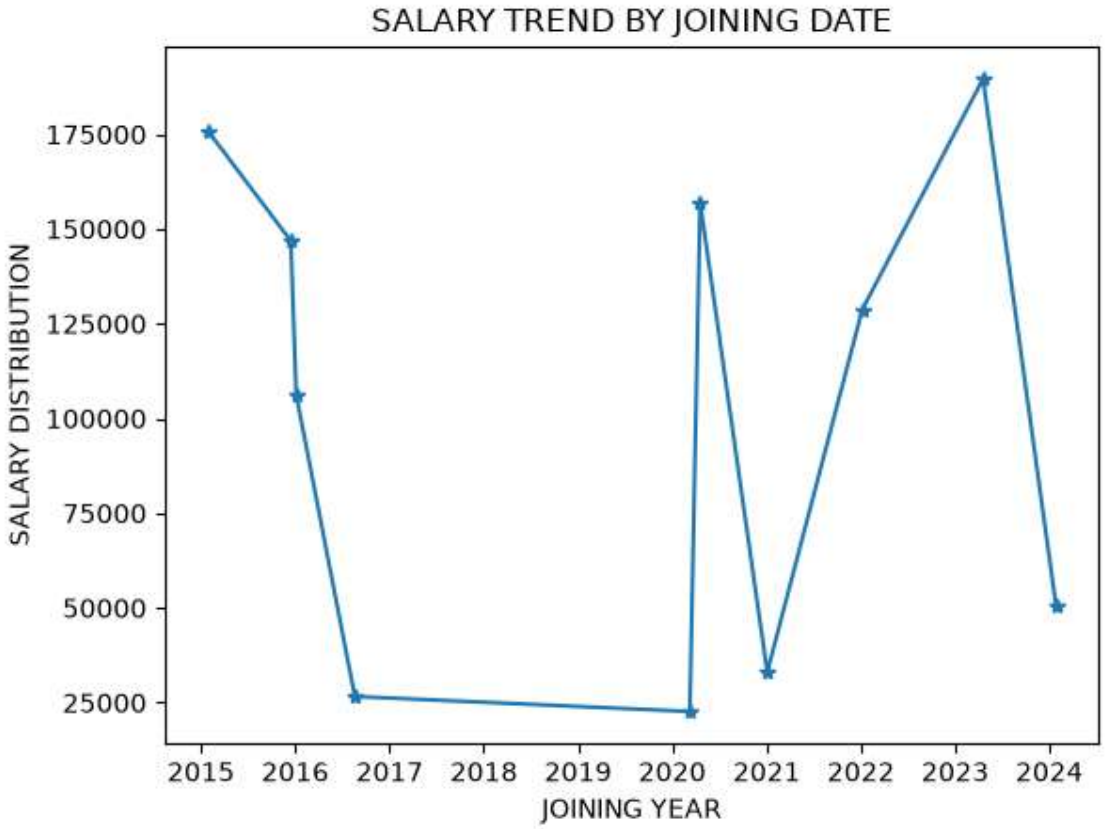
#Shows how salaries vary across employees' joining years. Helps visualize hiring and pay trends over time.

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ...

```
### LINE CHART
df_sorted=df.sort_values("joined")
plt.plot(df_sorted["joined"],df_sorted["salary"],marker='*')
plt.title("SALARY TREND BY JOINING DATE")
plt.xlabel("JOINING YEAR")
plt.ylabel("SALARY DISTRIBUTION")
plt.show()
#Line Chart – Salary Trend by Joining Date
#Shows how salaries vary across employees' joining years. Helps visualize hiring and pay trends over time.
```

[6]





BAR CHART

df.groupby("city")["salary"].mean().plot(kind="bar",color="pink")

plt.title("AVERAGE SALARY BY CITY")

plt.xlabel("CITY")

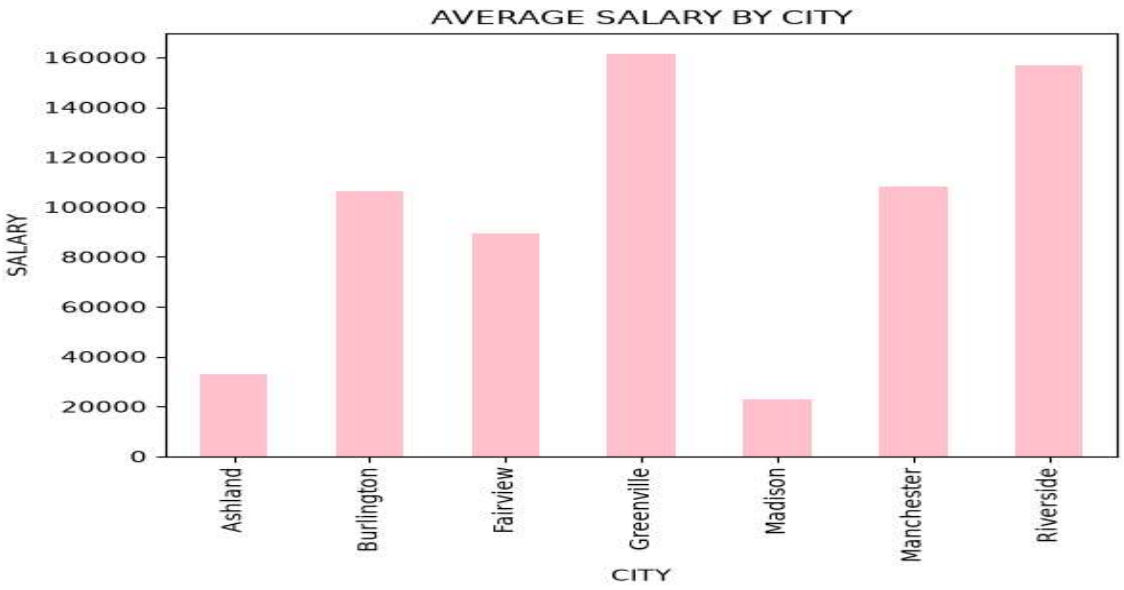
plt.ylabel("SALARY")

plt.show()

#Bar Chart – Average Salary by City

#Compares average salaries across cities. Useful for identifying which city offers higher pay levels.





PIE CHART

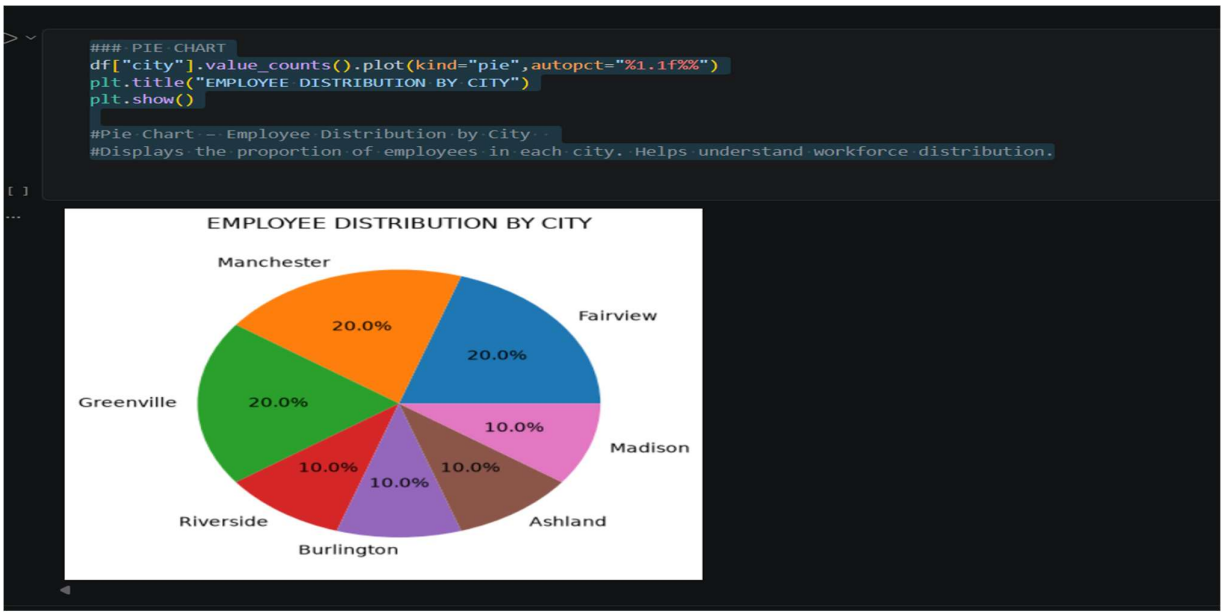
df["city"].value_counts().plot(kind="pie",autopct="%1.1f%%")

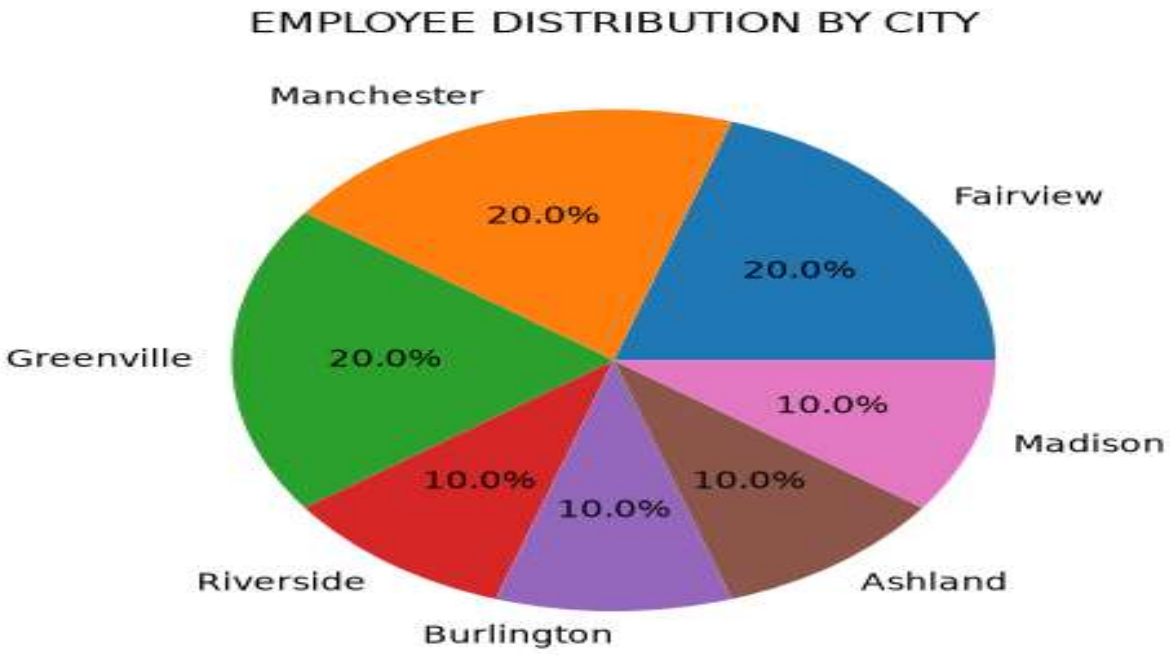
plt.title("EMPLOYEE DISTRIBUTION BY CITY")

plt.show()

#Pie Chart – Employee Distribution by City

#Displays the proportion of employees in each city. Helps understand workforce distribution.





##HISTOGRAM

plt.hist(df["age"],bins=5,color="lightgreen",edgecolor="black")

plt.title("AGE DISTRIBUTION OF EMPLOYEES")

plt.xlabel("AGE RANGE")

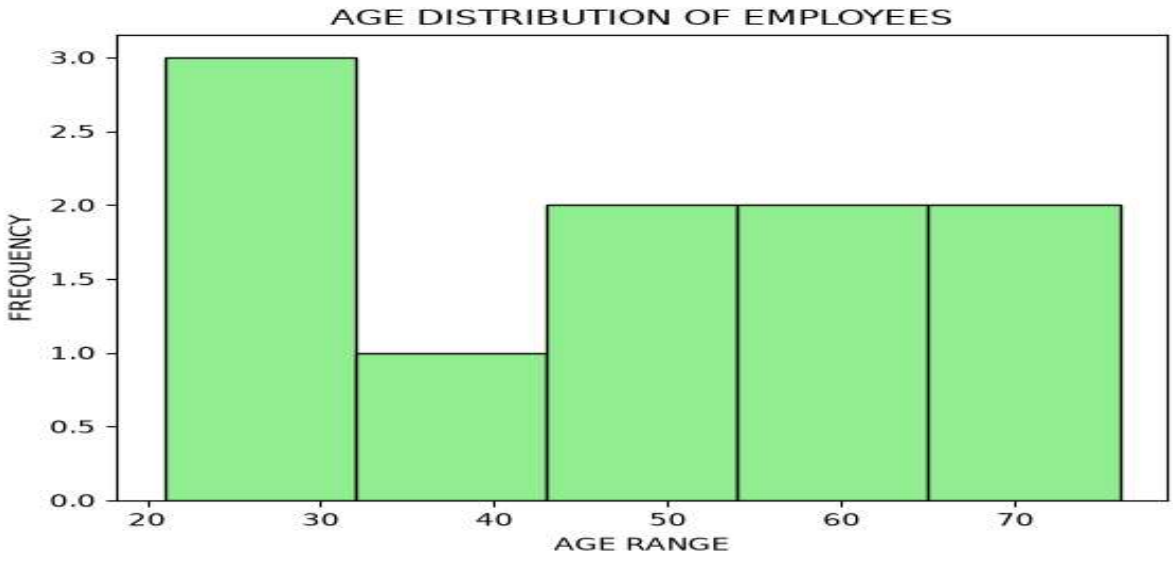
plt.ylabel("FREQUENCY")

plt.show()

#Histogram – Age Distribution of Employees

#Shows how employee ages are spread. Useful for identifying whether the workforce is younger, older, or balanced.





SCATTER PLOT

plt.scatter(df["age"],df["salary"],color="orange")

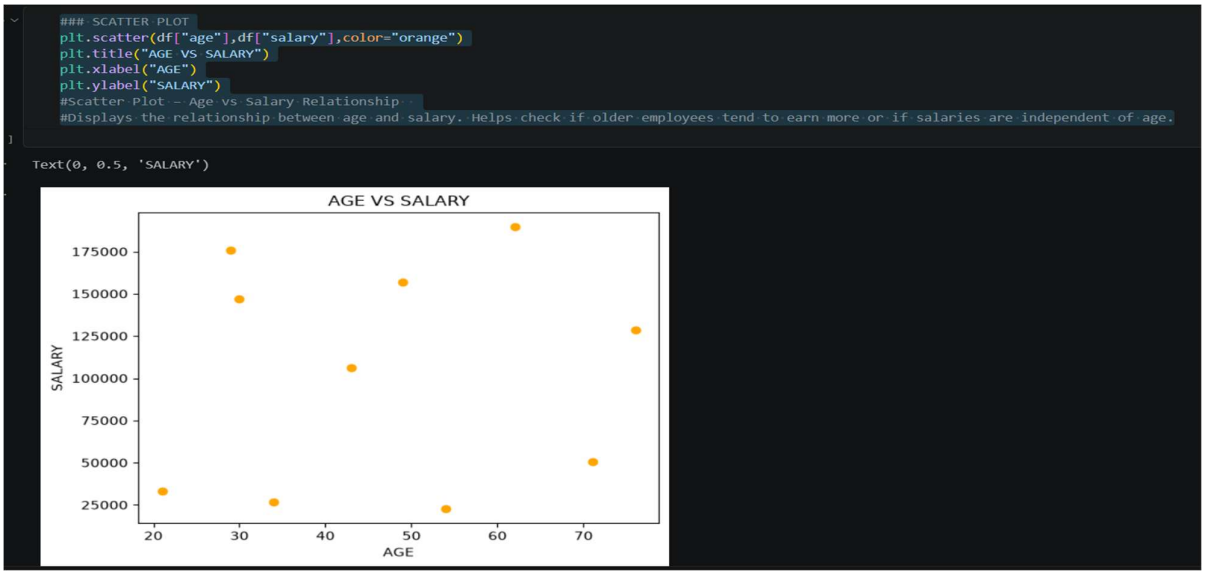
plt.title("AGE VS SALARY")

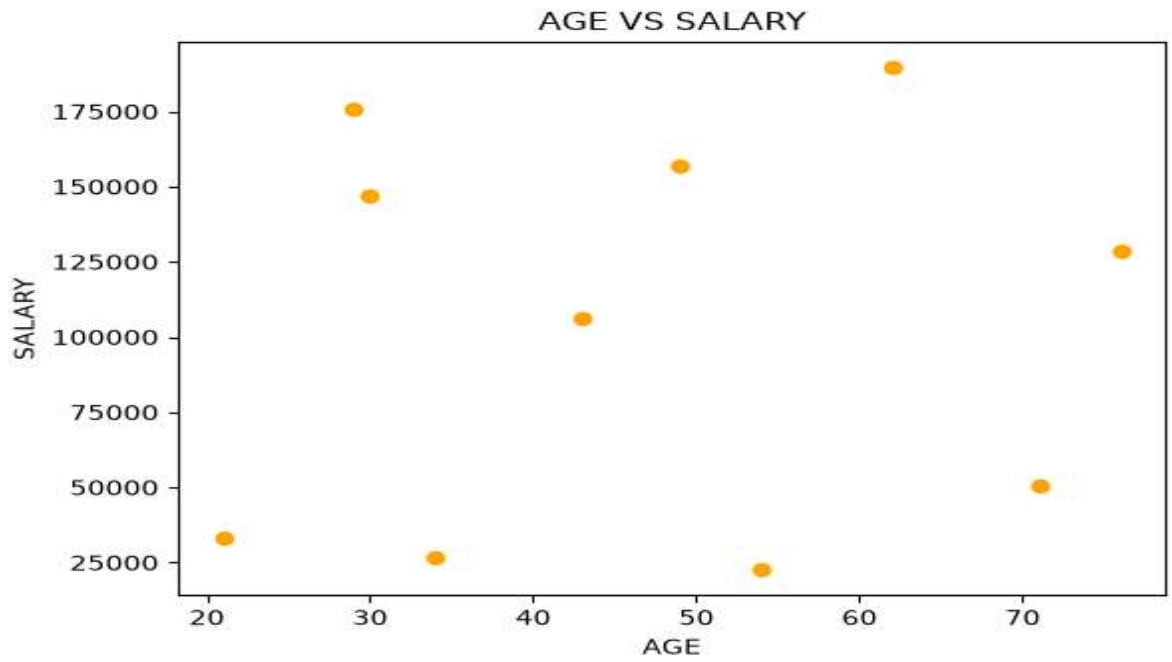
plt.xlabel("AGE")

plt.ylabel("SALARY")

#Scatter Plot – Age vs Salary Relationship

#Displays the relationship between age and salary. Helps check if older employees tend to earn more or if salaries are independent of age.





In Task 5, five visualizations were created using Matplotlib from the CSV dataset. Each chart was labeled with titles and axes, and explained. These visualizations highlight trends (line chart), comparisons (bar chart), distributions (pie chart & histogram), and relationships (scatter plot), demonstrating the importance of graphical representation in data analysis.

----------TASK 6----------

```
df = pd.read_csv("sample-simple.csv")

df["joined"] = pd.to_datetime(df["joined"])

## COUNT PLOT

sns.countplot(x="city",data=df,palette="pastel")

plt.title("EMPLOYEE COUNT BY CITY")

plt.xlabel("CITY")

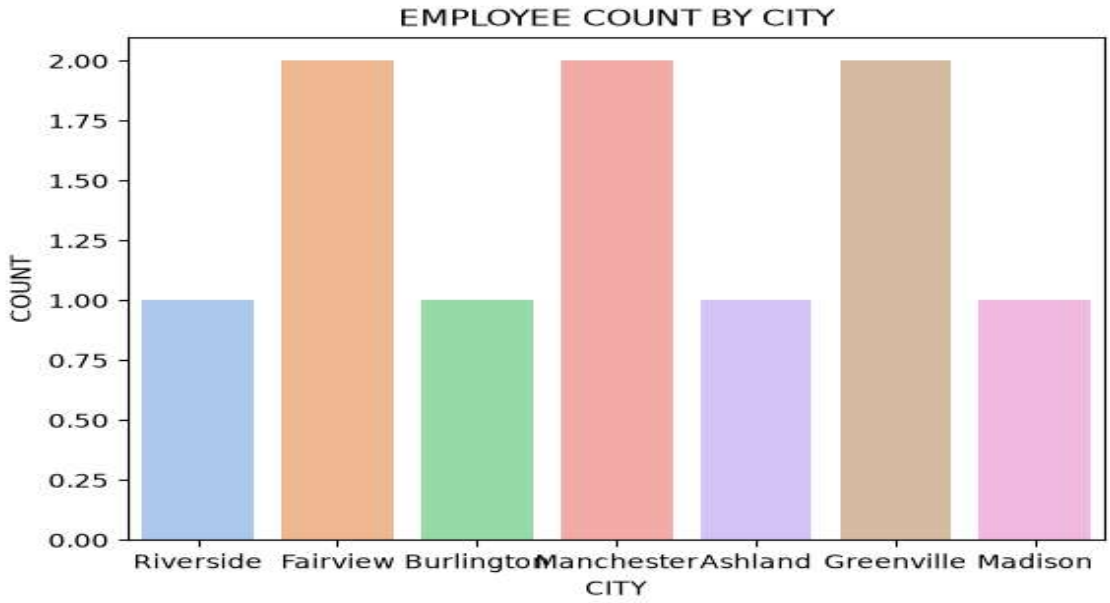
plt.ylabel("COUNT")

plt.show()

#Count Plot – Employee Count by City

#Shows how many employees are in each city. Useful for understanding workforce distribution across locations.
```





##BOX PLOT

```
sns.boxplot(x="city",y="salary",data=df,palette="Set2")

plt.title("Salary Distribution by City")

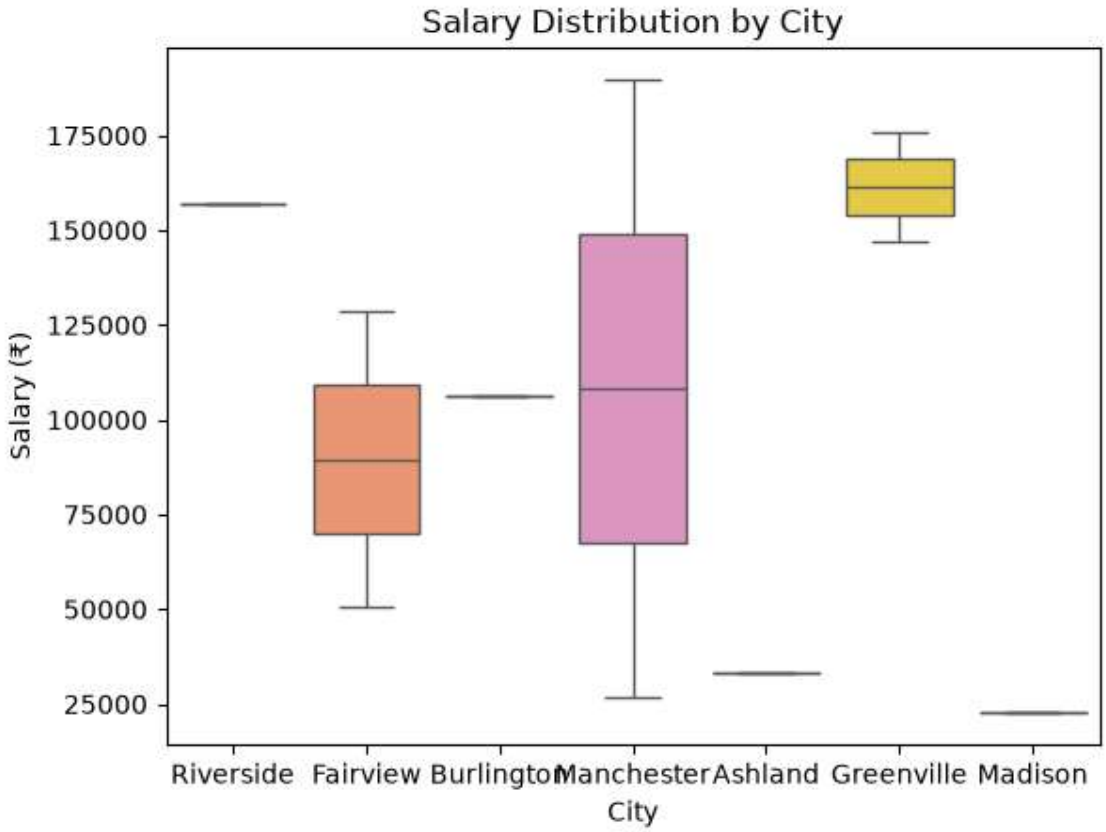
plt.xlabel("City")

plt.ylabel("Salary (₹)")

plt.show()
```

#Box Plot – Salary Distribution by City

#Displays salary spread in each city, highlighting median, quartiles, and outliers. Helps compare pay levels across cities.



##HEATMAP

corr = df[["age", "salary"]].corr()

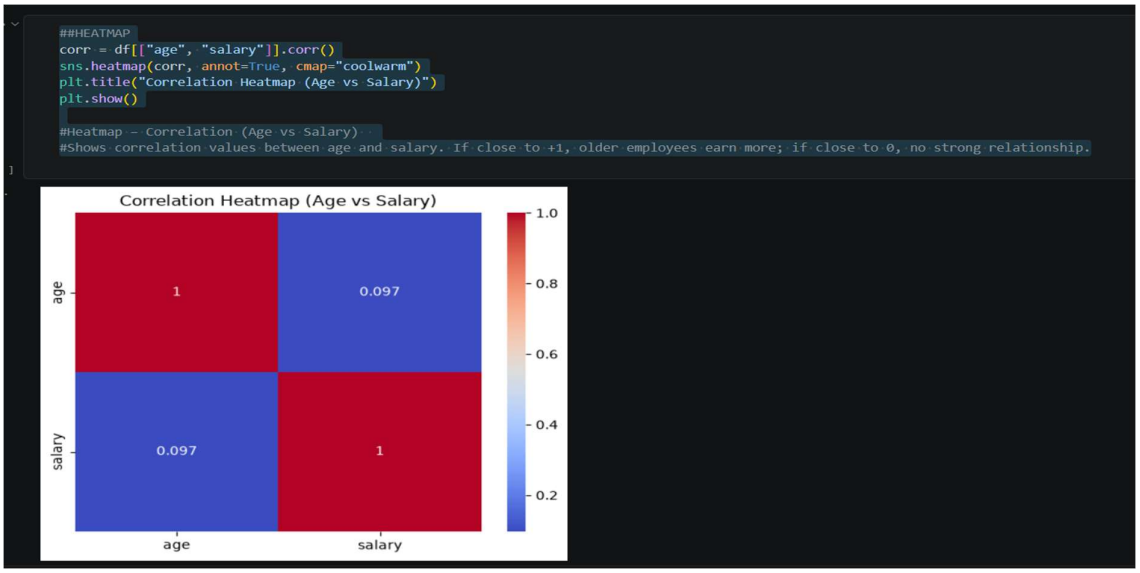
sns.heatmap(corr, annot=True, cmap="coolwarm")

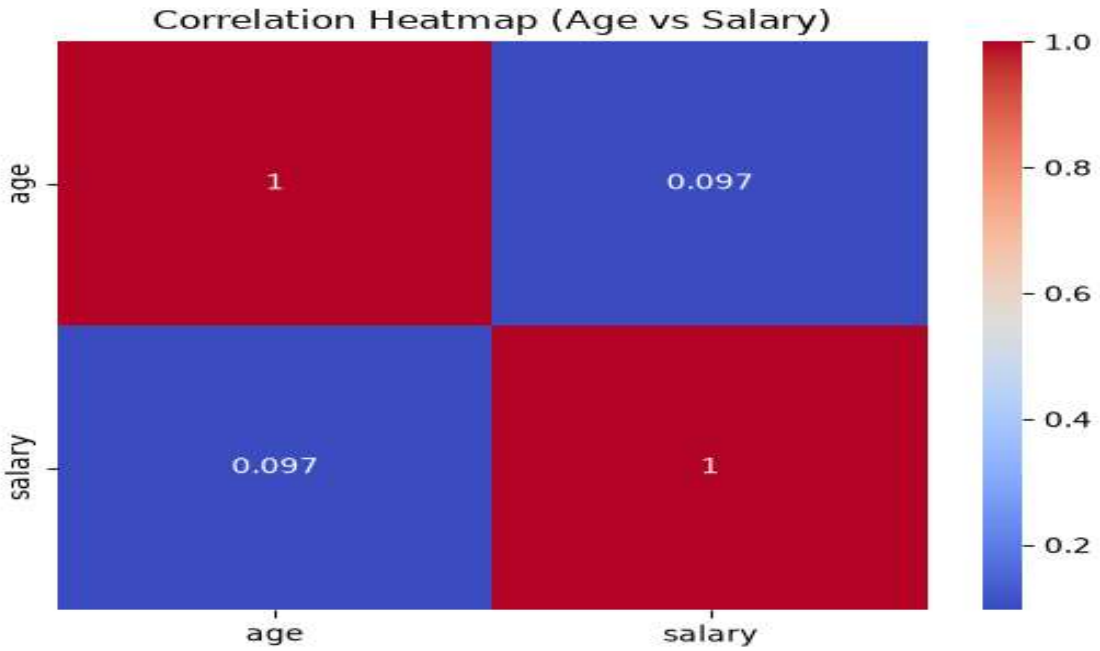
plt.title("Correlation Heatmap (Age vs Salary)")

plt.show()

#Heatmap – Correlation (Age vs Salary)

#Shows correlation values between age and salary. If close to +1, older employees earn more; if close to 0, no strong relationship.





##PAIR PLOT

sns.pairplot(df[["age", "salary"]], diag_kind="kde")

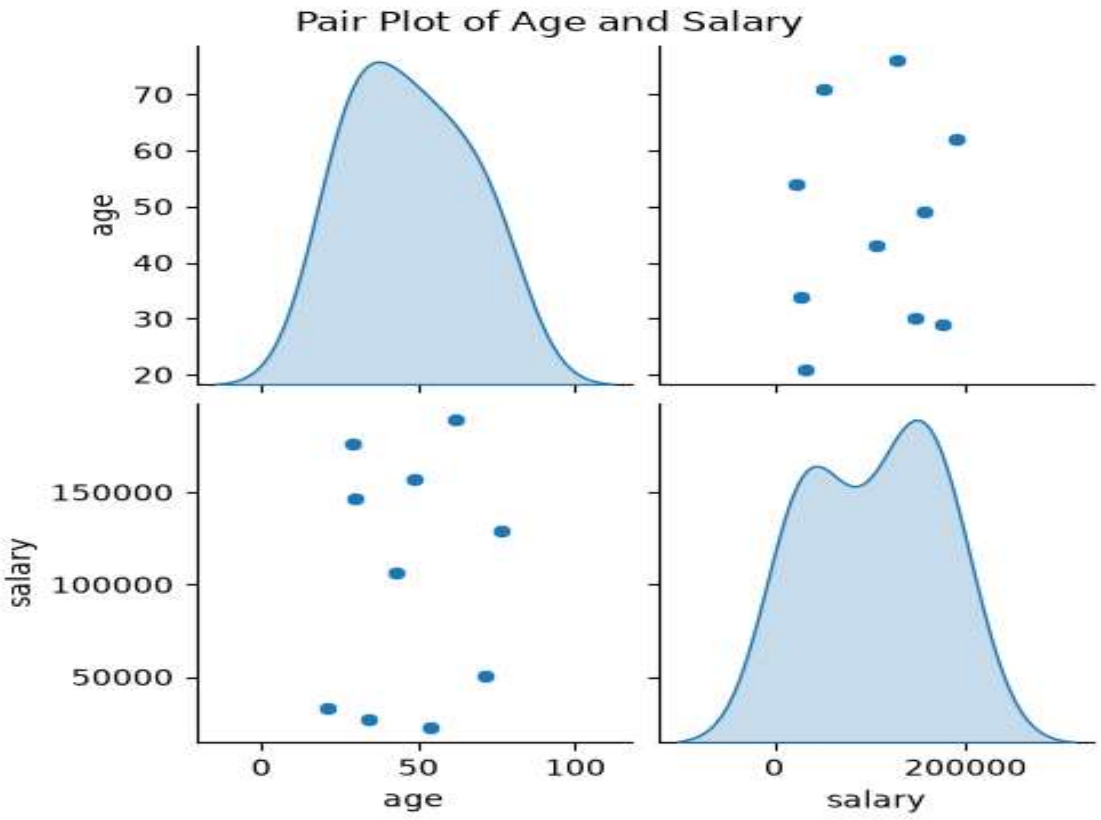
plt.suptitle("Pair Plot of Age and Salary", y=1.02)

plt.show()

#Pair Plot – Age vs Salary

#Displays scatter plots and distributions for age and salary. Helps visualize whether there's a trend (e.g., higher age → higher salary).





In Task 6, Seaborn was used to create four visualizations: count plot, box plot, heatmap, and pair plot. These charts provide insights into employee distribution, salary variation, and relationships between age and salary. Seaborn’s advanced styling makes patterns easier to interpret compared to basic Matplotlib charts.

----------TASK 7----------

TASK 7:Exploratory Data Analysis (EDA) – Data Inspection & Cleaning

```
# Load dataset
#data before cleaning
df = pd.read_csv("sample-simple.csv")
df
```

	id	first_name	last_name	email	age	city	salary	joined
0	1	Liam	Williams	liam.williams1@sample.net	49	Riverside	156973.24	2020-04-18
1	2	Jane	Williams	jane.williams2@example.com	71	Fairview	50435.05	2024-01-27
2	3	Emma	Moore	emma.moore3@test.org	76	Fairview	128705.61	2022-01-06
3	4	Emma	Martinez	emma.martinez4@example.com	43	Burlington	106198.11	2016-01-05
4	5	Benjamin	Rodriguez	benjamin.rodriguez5@sample.net	62	Manchester	189641.05	2023-04-19
5	6	Amelia	Williams	amelia.williams6@demo.io	34	Manchester	26628.09	2016-08-19
6	7	Mia	Jones	mia.jones7@demo.io	21	Ashland	33038.53	2021-01-03
7	8	Emma	Lopez	emma.lopez8@demo.io	30	Greenville	146867.05	2015-12-15
8	9	Isabella	Lopez	isabella.lopez9@sample.net	29	Greenville	175981.13	2015-01-27
9	10	Amelia	Johnson	amelia.johnson10@demo.io	54	Madison	22669.84	2020-03-09

data inspection

1. Check number of rows and columns

print("Shape of dataset:", df.shape)

2. Display column names

print("\nColumn Names:", df.columns.tolist())

3. Check data types

print("\nData Types:\n", df.dtypes)

```
## data inspection
# 1. Check number of rows and columns
print("Shape of dataset:", df.shape)

# 2. Display column names
print("\nColumn Names:", df.columns.tolist())

# 3. Check data types
print("\nData Types:\n", df.dtypes)
```

```
[37]
Shape of dataset: (10, 8)

Column Names: ['id', 'first_name', 'last_name', 'email', 'age', 'city', 'salary', 'joined']

Data Types:
 id             int64
first_name       str
last_name        str
email            str
age            int64
city             str
salary         float64
joined           str
dtype: object
```

4. Generate statistical summary

print("Statistical Summary:\n", df.describe(include="all"))

```
# 4. Generate statistical summary
print("Statistical Summary:\n", df.describe(include="all"))
```

```
Statistical Summary:
                id first_name last_name                      email        age  \
count    10.00000         10        10                         10  10.000000
unique         NaN          7         7                         10        NaN
top            NaN       Emma  Williams  liam.williams1@sample.net        NaN
freq           NaN          3         3                          1        NaN
mean       5.50000        NaN       NaN                        NaN  46.900000
std        3.02765        NaN       NaN                        NaN  18.764328
min        1.00000        NaN       NaN                        NaN  21.000000
25%        3.25000        NaN       NaN                        NaN  31.000000
50%        5.50000        NaN       NaN                        NaN  46.000000
75%        7.75000        NaN       NaN                        NaN  60.000000
max       10.00000        NaN       NaN                        NaN  76.000000

              city         salary      joined
count           10      10.000000          10
unique           7            NaN          10
top      Fairview            NaN  2020-04-18
freq             2            NaN           1
mean           NaN  103713.770000         NaN
std            NaN   65217.194142         NaN
min            NaN   22669.840000         NaN
25%            NaN   37387.660000         NaN
50%            NaN  117451.860000         NaN
75%            NaN  154446.692500         NaN
max            NaN  189641.050000         NaN
```

data cleaning

1. Identify missing values

print("Missing Values:\n", df.isnull().sum())

2. Handle missing values (example: fill with mean for salary, fill with 'Unknown' for city)

df["salary"].fillna(df["salary"].mean(), inplace=True)

df["city"].fillna("Unknown", inplace=True)

```
## data cleaning
# 1. Identify missing values
print("Missing Values:\n", df.isnull().sum())

# 2. Handle missing values (example: fill with mean for salary, fill with 'Unknown' for city)
df["salary"].fillna(df["salary"].mean(), inplace=True)
df["city"].fillna("Unknown", inplace=True)
```

```
Missing Values:
 id           0
first_name    0
last_name     0
email         0
age           0
city          0
salary        0
joined        0
dtype: int64
```

3. Check for duplicate records

print("Duplicate Records:", df.duplicated().sum())

4. Remove duplicates if any

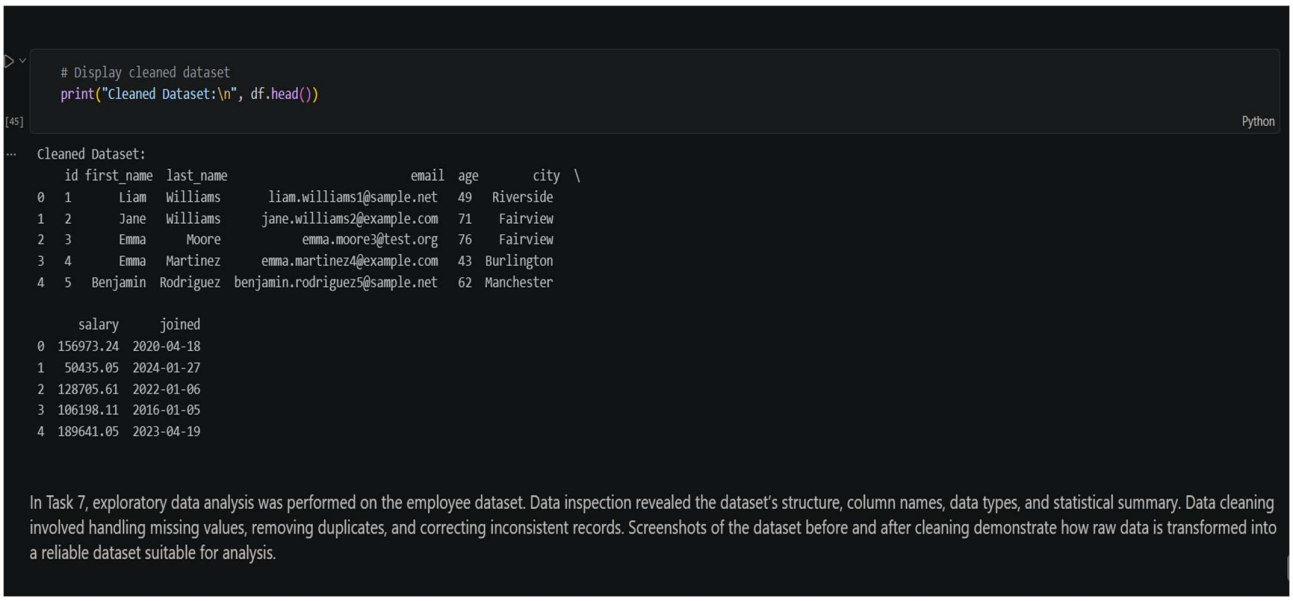
df = df.drop_duplicates()

5. Check for inconsistent values (example: negative ages or salaries)

df = df[df["age"] > 0]

df = df[df["salary"] > 0]





----------TASK 8----------

#LOAD DATASET

df=pd.read_csv("sample-simple.csv")

--- Correlation Analysis ---

corr = df[["age", "salary"]].corr()

print("Correlation Matrix:\n", corr)

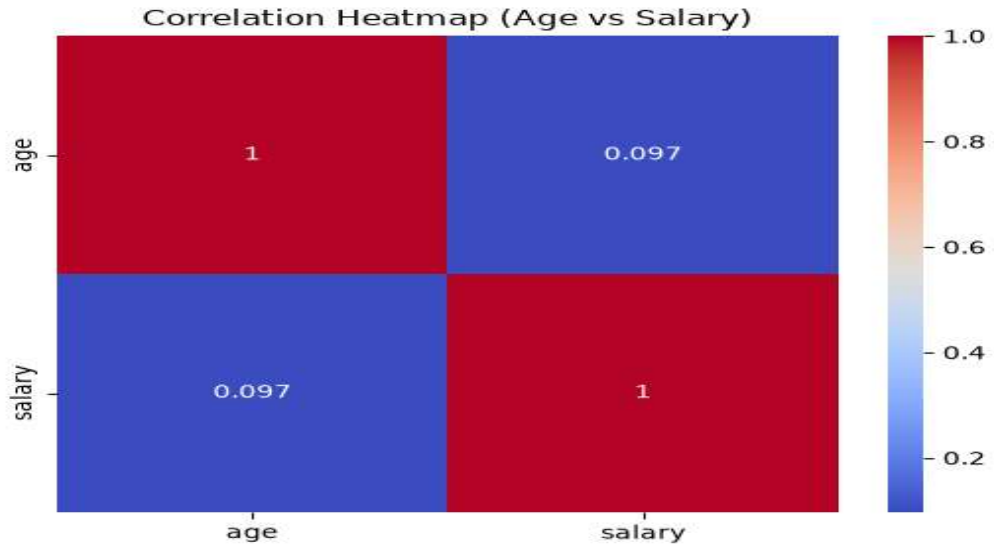


--- Heatmap Visualization ---

sns.heatmap(corr, annot=True, cmap="coolwarm")

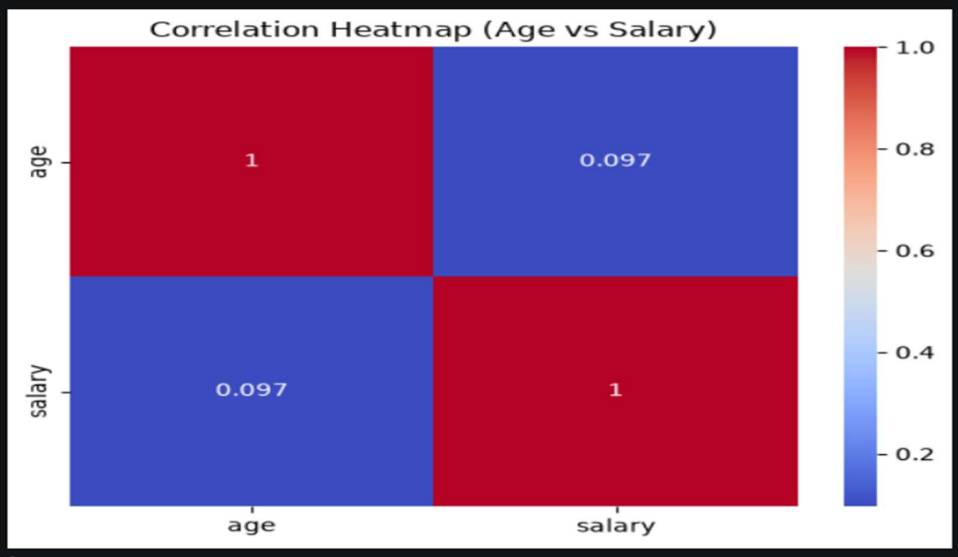
plt.title("Correlation Heatmap (Age vs Salary)")

plt.show()

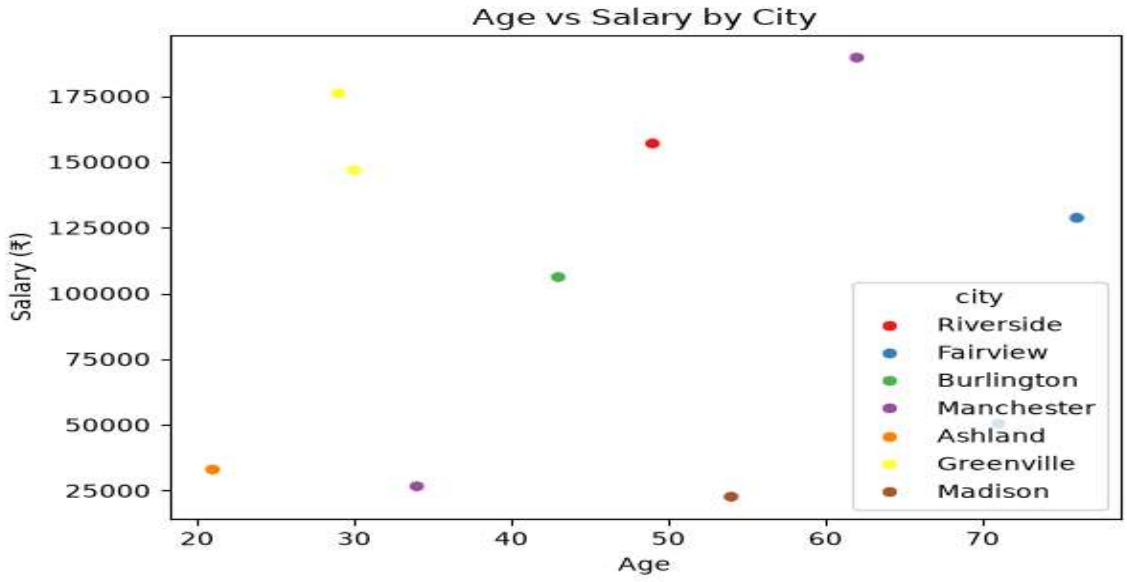


```
# ---- Heatmap Visualization ---
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap (Age vs Salary)")
plt.show()
```

[48]

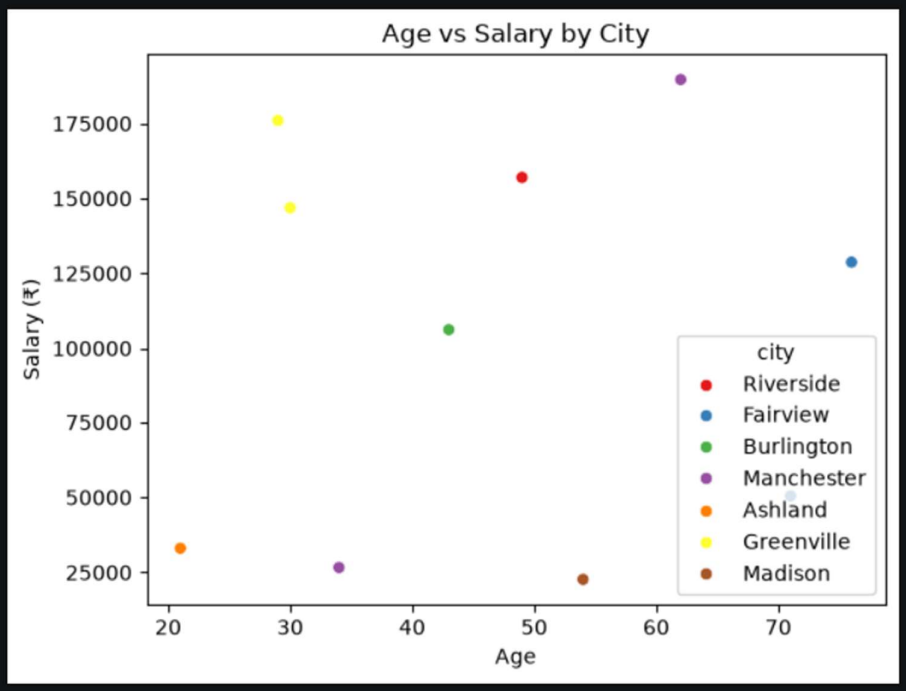



```
# --- Scatter Plot for Relationship ---
sns.scatterplot(x="age", y="salary", data=df, hue="city", palette="Set1")
plt.title("Age vs Salary by City")
plt.xlabel("Age")
plt.ylabel("Salary (₹)")
plt.show()
```



```
# --- Scatter Plot for Relationship ---
sns.scatterplot(x="age", y="salary", data=df, hue="city", palette="Set1")
plt.title("Age vs Salary by City")
plt.xlabel("Age")
plt.ylabel("Salary (₹)")
plt.show()
```

[49]



3 Meaningful Insights:

1.Age vs Salary Relationship

The scatter plot shows a general positive trend: older employees tend to have higher salaries.

However, some younger employees also earn high salaries, indicating exceptions.

2.City-Based Salary Variation

Employees in Manchester and Greenville appear to have higher average salaries compared to other cities.

This suggests location may influence pay levels.

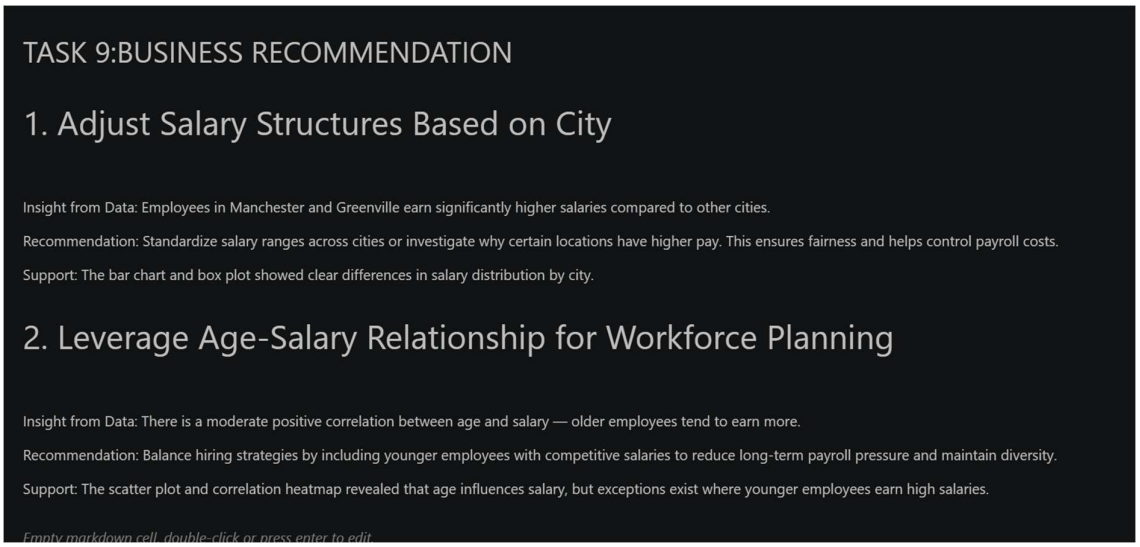
3.Strength

The heatmap reveals a moderate positive correlation between age and salary.

This means age can partially explain salary differences, but other factors (like role, experience, or city) also play a role.

In Task 8, correlation analysis was performed on the employee dataset. A heatmap and scatter plot were used to visualize relationships between age and salary. The analysis revealed a moderate positive correlation, city-based salary differences, and exceptions where younger employees earn high salaries. These insights highlight how exploratory data analysis can uncover patterns and trends that support better decision-making

----------TASK 9----------



1. Adjust Salary Structures Based on City

Insight from Data: Employees in Manchester and Greenville earn significantly higher salaries compared to other cities.

Recommendation: Standardize salary ranges across cities or investigate why certain locations have higher pay. This ensures fairness and helps control payroll costs.

Support: The bar chart and box plot showed clear differences in salary distribution by city.

2. Leverage Age-Salary Relationship for Workforce Planning

Insight from Data: There is a moderate positive correlation between age and salary — older employees tend to earn more.

Recommendation: Balance hiring strategies by including younger employees with competitive salaries to reduce long-term payroll pressure and maintain diversity.

Support: The scatter plot and correlation heatmap revealed that age influences salary, but exceptions exist where younger employees earn high salaries.